The Journal of Open Source Software

VCNet: Generation of realistic counterfactuals for real-world tabular data

Thomas Guyet [1¶], **Victor Guyomard** [2], **Tassadit Bouadi** [3], **and Françoise Fessant** [2]

1 Inria, AIstroSight team, Université Claude Bernard Lyon 1, Hospices Civils de Lyon, Lyon, France **2** Orange Research, Lannion, France **3** Univ Rennes, Inria, CNRS, IRISA, Rennes, France ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 16 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Machine learning models increasingly support decisions about individuals in domains such as healthcare, credit scoring or retail. In these applications, individuals are usually described by *tabular data*, i.e., records made of numerical and categorical attributes. Counterfactual explanations answer the question "what should be changed in this record for the model to make a different decision?" (Verma et al., 2024). Useful counterfactuals have to be *valid* (the model predicts a different class), *actionable* (they suggest few and small changes) and *realistic*: they must be plausible with respect to the data distribution and respect hard constraints, such as attributes that cannot be modified (e.g., protected attributes).

`VCNet` is a Python library for generating realistic counterfactual explanations for tabular data. It implements the VCNet family of models (Guyomard et al., 2023, 2026), in which a conditional variational autoencoder (Kingma & Welling, 2014; Sohn et al., 2015) learns to generate counterfactuals. Once trained, the generator produces a counterfactual in a single forward pass, which is much more efficient than optimization-based approaches that solve one optimization problem per explained example (Wachter et al., 2018). The library offers two training modes: a *joint-learning* mode, in which a predictor and a counterfactual generator are trained together, and a *post-hoc* mode, in which a generator is trained to explain an existing classifier implemented with `PyTorch` (Paszke et al., 2019), `scikit-learn` (Pedregosa et al., 2011) or `XGBoost` (Chen & Guestrin, 2016). Datasets are provided as pandas (McKinney, 2010) dataframes and the counterfactuals are returned as dataframes with the same structure, so that the explanations can be read directly by the users.

Statement of need

The VCNet method (Guyomard et al., 2023) was originally published with research code designed to reproduce benchmark experiments. Such code assumes that the data have already been encoded into numerical tensors, and it can explain only the classifier trained jointly with the generator. These limitations prevent its use on real-world datasets, where practitioners handle mixed attribute types, missing values and immutable attributes, and where the model to be explained already exists and is often not a neural network.

The `VCNet` library addresses these needs. It is intended for data scientists who want to add counterfactual explanations to a tabular data analysis pipeline, and for researchers in explainable AI who need an efficient and realistic counterfactual generator as a baseline or as a building block. Its main features are:

- a preprocessing pipeline, configured by a dictionary of settings, that scales numerical

DRAFT

attributes, imputes missing values and encodes categorical attributes, together with the reverse pipeline that turns generated counterfactuals back into readable records;
- the native handling of immutable attributes, which are never modified by the generator;
- a post-processing step that rounds the generated numerical values to the precision observed in the training data, which increases the realism of the counterfactuals;
- the support of binary and multi-class classification tasks, with configurable strategies to choose the target class of the counterfactuals, or an explicit target class;
- post-hoc explanation of `PyTorch` modules and of any probabilistic `scikit-learn` or `XGBoost` classifier, including non-differentiable models such as random forests.

State of the field

Several general-purpose Python libraries provide explanation methods. `Xplique` (Fel et al., 2022) implements a counterfactual method based on the nearest unlike neighbor (Dasarathy, 1995). `Alibi Explain` (Klaise et al., 2021) includes four counterfactual methods (counterfactual instances, contrastive explanations, prototype-guided counterfactuals and reinforcement-learning-based counterfactuals), and `AI Explainability 360` (Arya et al., 2019) provides contrastive explanations. These libraries are mainly designed for numerical data such as images and do not address the specific challenges of tabular data. `OpenXAI` (Agarwal et al., 2022) targets tabular data but does not include counterfactual explanations. `OmniXAI` (Yang et al., 2022) includes a model-agnostic counterfactual explainer for tabular data, but it does not explicitly handle immutability constraints.

Two libraries are dedicated to counterfactual generation for tabular data and are commonly used for benchmarking. `CARLA` (Pawelczyk et al., 2021) implements several generation methods together with a preprocessing pipeline that encodes categorical features and enforces immutability constraints, but it depends on a Python environment that is incompatible with recent versions of the data science libraries. `ReLax` (Guo et al., 2024) reimplements most of CARLA's functionality in JAX; it offers limited compatibility with `pandas` dataframes and cannot explain shallow classification models.

We chose to build a new library rather than contributing VCNet to one of these frameworks. CARLA is tied to outdated dependencies, and ReLax requires models written in JAX, whereas our goal is to explain the models that practitioners already use (`scikit-learn`, `XGBoost` or `PyTorch`) directly from `pandas` dataframes.

Software design

`VCNet` is organized around three components (Figure 1).

The `DataCatalog` class is a `Lightning` data module that manages a tabular dataset. It is configured with a dictionary that specifies the target attribute, the numerical, categorical and immutable attributes, and the preprocessing methods. Preparing the data splits the dataset, fits the preprocessing pipeline and completes the settings with the information required by the models (e.g., the size of the encoded features or the positions of the immutable features). The same object applies the reverse pipeline to the generated counterfactuals.

The models derive from the abstract class `VCNetBase`, a `Lightning` module (Falcon & The PyTorch Lightning team, 2019) that implements the conditional variational autoencoder, the training loop and the generation of counterfactuals. The generation conditions the decoder on a modified class probability vector and on the immutable features, which are copied unchanged from the factual example. The `VCNet` class adds shared layers and a predictor that are trained jointly with the generator, whereas the `PHVCNet` class takes an already trained classifier whose parameters are frozen. The `SKLearnClassifier` wrapper exposes `scikit-learn` and `XGBoost`

DRAFT

classifiers through the interface expected by `PHVCNet`. Since the generator only needs the predicted class probabilities during training, non-differentiable classifiers can be explained.

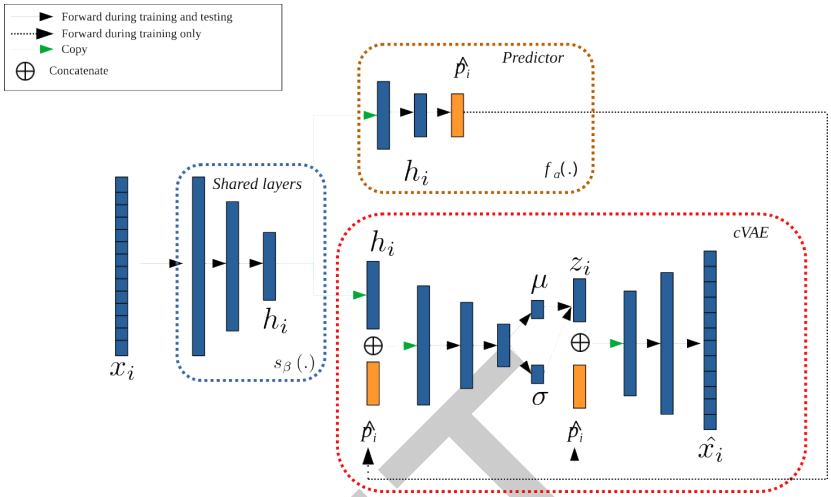


Figure 1: Architecture of the joint-learning VCNet model: shared layers feed a predictor and a conditional variational autoencoder (cVAE), conditioned by the predicted class probabilities.

We made two main design choices. First, all hyperparameters (dataset settings and model parameters) are gathered in a single dictionary. This follows the practices of `Lightning` and makes the library easy to combine with hyperparameter optimization frameworks, at the cost of less static checking than typed configuration objects. Second, the models are implemented as `Lightning` modules rather than with a custom training loop, so that users benefit from callbacks, logging and hardware acceleration; for instance, the library provides a monotonic annealing callback that progressively increases the weight of the Kullback-Leibler loss during training. New counterfactual generators can be implemented by subclassing `VCNetBase` and overriding its pre-encoding and classification methods.

The following example trains a joint-learning model on the *Adult* dataset and generates counterfactuals as a dataframe:

```
import pandas as pd
import lightning as L
from vcnet import DataCatalog, VCNet

hp = {
    "dataset": {
        "target": "income",
        "continuous": ["age", "hours-per-week"],
        "categorical": ["workclass", "education", "occupation", "race", "gender"],
        "immutables": ["race", "gender"],
        "activate_rounding": True,
    },
    "vcnet_params": {"lr": 2e-3, "epochs": 5, "lambda_KLD": 0.5, "lambda_CE": 0.93,
                     "lambda_BCE": 1, "latent_size": 19, "latent_size_share": 304,
                     "mid_reduce_size": 152},
}
dataset = DataCatalog(hp["dataset"])
hp["dataset"] = dataset.prepare_data(pd.read_csv("adult.data"))

vcnet = VCNet(hp)
```

DRAFT

```
L.Trainer(max_epochs=5).fit(vcnet, train_dataloaders=dataset.train_dataloader())

for data, _ in dataset.test_dataloader():
    cf, cf_proba = vcnet.counterfactuals(data)
    print(dataset.data_unloader(cf, cf_proba))  # counterfactuals as a dataframe
```

The library is distributed on PyPI, is tested with `pytest` in a continuous integration pipeline, and is documented online with installation instructions, a user guide, examples (multi-class problems, post-hoc explanation of `scikit-learn` classifiers, a study of the plausibility of the generated counterfactuals) and an API reference.

Research impact statement

`VCNet` has been used to conduct the experiments of the journal article that extends the VCNet method with the handling of immutable features (Guyomard et al., 2026). It has also been used by external users who benchmarked their counterfactuals generation approaches against our state-of-the-art method (Guo et al., 2023; Markou et al., 2026). `VCNet` has been also used in the applied contexts of cybersecurity (Galwaduge & Samarabandu, 2025). Dominici et al. reused our library for implementing counterfactual concept bottleneck models (Dominici et al., 2025). Finally, it is also used as a teaching material for counterfactual explanations in machine learning courses.

AI usage disclosure

The core algorithms and the software architecture were designed and implemented by the authors. Claude Opus 5.5 (Anthropic), used through Claude Code, assisted with the reorganization and proofreading of the documentation and docstrings. The authors reviewed, edited and validated all AI-assisted outputs, and made all the design decisions of the software.

Acknowledgements

The Victor Guyomard's PhD has been founded by the Orange Company.

References

Agarwal, C., Krishna, S., Saxena, E., Pawelczyk, M., Johnson, N., Puri, I., Zitnik, M., & Lakkaraju, H. (2022). OpenXAI: Towards a transparent evaluation of model explanations. *Advances in Neural Information Processing Systems*, *35*, 15784–15799. https://proceedings.neurips.cc/paper_files/paper/2022/hash/65398a0eba88c9b4a1c38ae405b125ef-Abstract-Datasets_and_Benchmarks.html

Arya, V., Bellamy, R. K. E., Chen, P.-Y., Dhurandhar, A., Hind, M., Hoffman, S. C., Houde, S., Liao, Q. V., Luss, R., Mojsilović, A., Mourad, S., Pedemonte, P., Raghavendra, R., Richards, J., Sattigeri, P., Shanmugam, K., Singh, M., Varshney, K. R., Wei, D., & Zhang, Y. (2019). *One explanation does not fit all: A toolkit and taxonomy of AI explainability techniques*. https://doi.org/10.48550/arXiv.1909.03012

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. https://doi.org/10.1145/2939672.2939785

Dasarathy, B. V. (1995). Nearest unlike neighbor (NUN): An aid to decision confidence estimation. *Optical Engineering*, *34*(9), 2785–2792. https://doi.org/10.1117/12.210755

DRAFT

Dominici, G., Barbiero, P., Giannini, F., Gjoreski, M., Marra, G., & Langheinrich, M. (2025). Counterfactual concept bottleneck models. *International Conference on Learning Representations*, *2025*, 5184–5207.

Falcon, W., & The PyTorch Lightning team. (2019). *PyTorch Lightning*. https://doi.org/10.5281/zenodo.3828935

Fel, T., Hervier, L., Vigouroux, D., Poche, A., Plakoo, J., Cadene, R., Chalvidal, M., Colin, J., Boissin, T., Bethune, L., Picard, A., Nicodeme, C., Gardes, L., Flandin, G., & Serre, T. (2022). Xplique: A deep learning explainability toolbox. *Workshop on Explainable Artificial Intelligence for Computer Vision (CVPR)*. https://doi.org/10.48550/arXiv.2206.04394

Galwaduge, V., & Samarabandu, J. (2025). Novel actionable counterfactual explanations for intrusion detection using diffusion models. *Journal of Cybersecurity and Privacy*, *5*(3). https://doi.org/10.3390/jcp5030068

Guo, H., Nguyen, T. H., & Yadav, A. (2023). Counternet: End-to-end training of prediction aware counterfactual explanations. *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 577–589.

Guo, H., Xiong, X., Zhang, W., & Yadav, A. (2024). ReLax: Efficient and scalable recourse explanation benchmarking using JAX. *Journal of Open Source Software*, *9*(103), 6567. https://doi.org/10.21105/joss.06567

Guyomard, V., Fessant, F., Bouadi, T., & Guyet, T. (2026). Generating efficiently realistic counterfactual explanations. *Machine Learning*, *115*(2), 27. https://doi.org/10.1007/s10994-025-06947-2

Guyomard, V., Fessant, F., Guyet, T., Bouadi, T., & Termier, A. (2023). VCNet: A self-explaining model for realistic counterfactual generation. *Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2022)*, *13713*, 437–453. https://doi.org/10.1007/978-3-031-26387-3_27

Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. *International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.1312.6114

Klaise, J., Van Looveren, A., Vacanti, G., & Coca, A. (2021). Alibi Explain: Algorithms for explaining machine learning models. *Journal of Machine Learning Research*, *22*(181), 1–7. http://jmlr.org/papers/v22/21-0017.html

Markou, K., Kalogeraki, V., & Gunopulos, D. (2026). FCx: An algorithm for finding feasible counterfactual explanations. *arXiv Preprint arXiv:2609.19383*.

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. https://doi.org/10.25080/Majora-92bf1922-00a

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*, 8024–8035. https://doi.org/10.48550/arXiv.1912.01703

Pawelczyk, M., Bielawski, S., Heuvel, J. van den, Richter, T., & Kasneci, G. (2021). CARLA: A python library to benchmark algorithmic recourse and counterfactual explanation algorithms. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*. https://doi.org/10.48550/arXiv.2108.00783

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830. http://jmlr.org/papers/v12/pedregosa11a.html

DRAFT

Sohn, K., Lee, H., & Yan, X. (2015). Learning structured output representation using deep conditional generative models. *Advances in Neural Information Processing Systems*, *28*. https://papers.nips.cc/paper/5775-learning-structured-output-representation-using-deep-conditional-generative-models

Verma, S., Boonsanong, V., Hoang, M., Hines, K., Dickerson, J., & Shah, C. (2024). Counterfactual explanations and algorithmic recourses for machine learning: A review. *ACM Computing Surveys*, *56*(12). https://arxiv.org/abs/2010.10596

Wachter, S., Mittelstadt, B. D., & Russell, C. (2018). Counterfactual explanations without opening the black box: Automated decisions and the GDPR. *Harvard Journal of Law and Technology*, *31*(2), 841–887. https://doi.org/10.2139/ssrn.3063289

Yang, W., Le, H., Savarese, S., & Hoi, S. (2022). OmniXAI: A library for explainable AI. *arXiv Preprint arXiv:2206.01612*. https://doi.org/10.48550/arXiv.2206.01612

DRAFT